

编译原理-语义分析 03

语法制导翻译

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 5 月 7 日



福州大学

FUZHOU UNIVERSITY

1958



① 知识点回顾

② 语法制导翻译 (Syntax-Directed Translation, SDT)

符号表管理

语法制导翻译

语法制导定义



- ① 知识点回顾
 - 语义分析概述
 - 符号表管理
 - 语法制导翻译
 - 语法制导定义

- ② 语法制导翻译 (Syntax-Directed Translation, SDT)

① 静态语义检查:

- **目的：**在编译阶段识别不符合语言规范的逻辑错误，避免程序运行时崩溃或不可

- ## ② 构建符号表 (Symbol Table) ;

- ③ 为中间代码生成或优化提供语义信息;

- ¹类型系统、作用域规则等

函数调用等操作的合法性（如整型与浮点型混
用可见性（如局部变量覆盖全局变量）。
类型匹配、结构体成员访问合法性等。
循环 / 条件语句的合理结构。
语义信息（如类型、内存地址）。

- 1 类型检查：验证表达式、赋值、函数调用等操作的合法性（如整型与浮点型混合运算的隐式转换）。
- 2 作用域分析：解析变量 / 函数的可见性（如局部变量覆盖全局变量）。
- 3 一致性检查：函数参数数量与声明匹配、结构体成员访问合法性等。
- 4 控制流验证：检测不可达代码、循环 / 条件语句的合理结构。
- 5 符号表管理：动态维护标识符的语义信息（如类型、内存地址）。

语义分析概述

符号表管理

语法制导翻译

语法制导定义

核心结构与功能

符号表管理是编译过程中维护程序语义正确性的核心机制。其核心结构为**键值对 (Key-Value)**。以标识符名称（变量、函数名、类型名、类名等）为键，关联类型、作用域、内存地址等属性。

符号表的功能（即操作接口）：

- ① 插入（声明时注册符号）；
- ② 查询（引用时验证存在性）；
- ③ 删除（退出作用域时清理）。

特别地，这些操作都需要高效进行²。

²需要使用哈希表、二叉查找树、可持久化数据结构等

符号表的实现

- 如何管理键值对？
 - 数据结构：二叉查找树（红黑树、AVL 树）、哈希表；
 - 现成的容器：Python 的字典、Java 的 Map (TreeMap 和 HashMap)、C++ 的 map 和 unordered_map；
`HashMap<String, SymbolBase> table;`
- 如何解析、管理作用域？
 - 理想中的方案：可持久化数据结构；
 - 写时复制 (Copy-on-Write, COW)；
 - 可以零成本恢复历史版本；
 - 用空间换时间；
 - 现实中的方案：
 - 顶层浅拷贝：**注意和深度拷贝的区别**；
 - 记录新增和变化量，离开时还原；



① 知识点回顾

语义分析概述

符号表管理

语法制导翻译

语法制导定义

② 语法制导翻译 (Syntax-Directed Translation, SDT)

语法制导翻译 (Syntax-Directed Translation)

定义 语法制导翻译使用上下文无关文法 CFG (context free grammar) 来引导对语言的翻译，是一种面向文法的翻译技术。

所处阶段 语法制导翻译横跨了**语法分析、语义分析、中间代码生成**三个阶段。

基本思想 当归约 (或推导) 到某个产生式时，除了按照产生式进行相应的代换之外 (语法分析)，还要按照产生式所对应的语义规则执行相应的语义动作，如计算表达式的值、产生中间代码 (语义分析)。

属性 (如类型 T 、值 val 等), 用来表示语法成分

的**语法分析树**, 并利用与产生式相关联的**语义规则**的文法符号的**语义属性值**。

则进行属性计算。

动作。

1) 联系起来有两种方式:

- Semantic Definitions, SDD)
- Directed Translation Scheme, SDT)

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

语义分析概述
符号表管理
语法制导翻译
语法制导定义

语法制导定义 (Syntax-Directed Definitions)

- SDD 是对 CFG 的推广
- SDD 为每个文法符号 (终结符或非终结符) 配备若干相关的“值” (称为属性), 代表与文法符号相关信息, 如类型、值、代码序列、符号表内容等。
- SDD 为每个文法的每个产生式配备一组属性的语义规则, 对属性进行计算和传递。
- 如果 X 是一个**文法符号**, a 是 X 的一个**属性**, 则用 $X.a$ 表示**属性** a 在某个标号为 X 的**分析树结点**上的**值**。
- 文法符号的属性包括:
 - 综合属性 (Synthesized Attribute)
 - 继承属性 (Inherited Attribute)

- 例

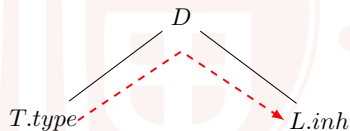
15 / 58

继承属性

- 在分析树结点 N 上的非终结符 A 的**继承属性**只能通过 **N 的父结点、 N 的兄弟结点或 N 本身**的属性值来定义。

例

产生式	语义规则
$D \rightarrow T L$	$L.inh = T.type$



- 终结符没有继承属性**。终结符从**词法分析器**处获得的属性值被归为综合属性值。

- 对出现在产生式**右边**的**继承属性**和出现在产生式**左边**的**综合属性**都必须提供一个**计算规则**。属性计算规则只能使用本产生式中的文法符号的属性。

例

对于产生式 $F \rightarrow digit$, 左侧文法符号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到。

- 出现在产生式**右边**的**综合属性**和出现在产生式**左边**的**继承属性**不由该产生式所给的属性计算规则进行计算, 由**其他产生式**的属性计算规则或者由属性计算器的参数提供。
- 语义规则所描述的工作可以包括**属性计算**、**静态语义检查**、**符号表操作**、**代码生成**等。

对于产生式 $F \rightarrow digit$, 左侧文法符号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到。

- 授课教师：柯嵩宇

属性。对于给定的输入串 x ，若要应用语义规则

1. 计算非终结符 A 的每个属性值，必须先计算

依赖关系，在对语法分析树节点的一个属性求值之前，必须先计算该节点的所有属性值。

先给定。

SDD 为 CFG 中的文法符号设置语义属性。对于给定的输入串 x , 若要应用语义规则计算分析树中各结点对应的属性值则

- 应按照什么顺序计算属性值？
 - 语义规则建立了属性之间的依赖关系，在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。
 - 文法开始符号的继承属性应事先给定。

依赖图 (Dependency Graph)

- 语法树中各结点的综合属性和继承属性之间的相互依赖关系可以用**依赖图**（有向图）来描述。
- 依赖图指出语法树中各结点属性值的计算顺序。
- 特点：先有文法，后确定语义计算顺序。

文法的良定义 (Well-defined)

- 如果一个 SDD 中不存在属性之间的**循环依赖关系** (即依赖图中不存在环), 则该文法是**良定义 (Well-defined)** 的, 依赖图存在拓扑排序。
- 一个依赖图的任何拓扑排序都给出一棵语法树中结点的语义规则计算的有效顺序。

SDD 求值顺序

- 对于只包含综合属性的 SDD，属性值的计算正好和自底向上分析建立分析树结点同步进行
- 对于同时具有继承属性和综合属性的 SDD，不能保证一定存在一个顺序来计算所有的属性值。
- 从计算的角度看，给定一个 SDD，很难确定是否存在某棵语法分析树，使得 SDD 的属性之间存在循环依赖关系，但可以给出一组充分条件，满足这组充分条件的 SDD 是无循环依赖的。

例 (S-SDD 示例)

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义**、**S-SDD**。

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

S-属性定义

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义**、**S-SDD**。

例 (S-SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

S-属性定义

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义**、**S-SDD**。
- 如果一个 SDD 是 S 属性的，可以按照语法分析树节点的任何自底向上顺序来计算它的各个属性值。

例 (S-SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

S-属性定义

- 仅仅使用**综合属性**的 SDD 称为**S 属性的 SDD**，或**S-属性定义**、**S-SDD**。
- 如果一个 SDD 是 S 属性的，可以按照语法分析树节点的任何自底向上顺序来计算它的各个属性值。
- S-属性定义可以在自底向上的语法分析过程中实现。

例 (S-SDD 示例)

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

L-属性定义

- L-属性定义 (也称为 L 属性的 SDD 或 L-SDD) 的直观含义: 在一个产生式所关联的各属性之间, 依赖图的边可以从左到右, 但不能从右到左 (因此取 Left 的首字母, 称为 L 属性)。
- 一个 SDD 是 L-属性定义, **当且仅当** 它的每个属性要么是一个**综合属性**, 要么是满足如下条件的**继承属性**: 假设存在一个产生式 $A \rightarrow X_1 X_2 \dots X_n$, 其右部符号 $X_i (1 \leq i \leq n)$ 的继承属性仅依赖于下列属性:
 - A 的**继承属性**;
 - 产生式中 X_i 左边的符号 $X_1, X_2, \dots, X_{i-1}$ 的属性;
 - X_i 本身的属性, 但 X_i 的全部属性不能在依赖图中形成环路。
- 特别地, 每个 S-属性定义都是 L-属性定义。



① 知识点回顾

② 语法制导翻译 (Syntax-Directed Translation, SDT)

S-SDT

L-SDT

自顶向下翻译

递归下降分析器

- SDD 方案只给出属性计算的定义，没有属性计算的次序等实现细节。
- 语法制导翻译方案 (SDT) 是 SDD 的补充，它给出了语义规则计算的次序，把实现细节表示出来。
- SDT 在产生式右部的合适位置嵌入了程序片段（用在 bison 等程序中使用 {} 包围，称为语义动作）的 CFG，并在语法分析的同时执行这些语义动作。

语法制导翻译示例

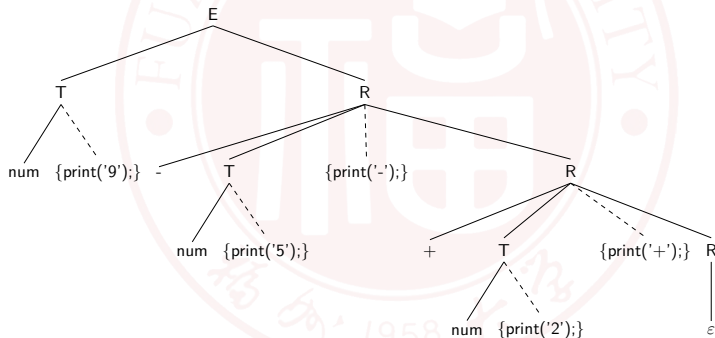
例

$$\begin{aligned} D &\rightarrow T \{ L.inh = T.type \} L \\ T &\rightarrow \text{int} \{ T.type = \text{int} \} \\ T &\rightarrow \text{real} \{ T.type = \text{real} \} \\ L &\rightarrow \{ L1.inh = L.inh \} L1, \text{id} \end{aligned}$$

SDT 综合示例

例 (表达式转换)

$E \rightarrow T R$
 $R \rightarrow + T \{ \text{print('+');} \} R \mid$
 $\quad - T \{ \text{print('-') ;} \} R \mid$
 $\quad \epsilon$
 $T \rightarrow \text{num} \{ \text{print(num.val);} \}$



SDT 的设计原则

- SDT 类似于 SDD, SDT 可以看作是 SDD 的具体实施方案, 它的语义规则的**计算顺序是显式给出的**, 这样就可以把具体的实现细节表示出来。
- SDT 的设计原则: 保证语义动作不会引用还没有计算的属性值。
 - **L-属性定义**: 本身就能确保每个动作不会引用尚未计算出来的属性。
 - **S-属性定义**: 为每一个语义规则建立一个包含赋值的动作, **并把这个动作放在相应的产生式右边的末尾**。

错误示例

例

$$\begin{aligned} S &\rightarrow A1 \{S.s=A1.s+A2.s\} A2 \\ A &\rightarrow a \{A.s=1\} \end{aligned}$$

错误示例

例

$$\begin{aligned} S &\rightarrow A1 \{S.s=A1.s+A2.s\} A2 \\ A &\rightarrow a \{A.s=1\} \end{aligned}$$

综合属性应该放在产生式的最后!!!

错误示例

例

$$\begin{aligned} S &\rightarrow A1 \{S.s=A1.s+A2.s\} A2 \\ A &\rightarrow a \{A.s=1\} \end{aligned}$$

综合属性应该放在产生式的最后!!!

$$\begin{aligned} S &\rightarrow A1 A2 \{S.s=A1.s+A2.s\} \\ A &\rightarrow a \{A.s=1\} \end{aligned}$$

SDD 的实现

- 如何使用 SDT 来实现两类重要的 SDD?



SDD 的实现

- 如何使用 SDT 来实现两类重要的 SDD?
- SDT 可以在语法分析过程中实现, 但有一定限制:



SDD 的实现

- 如何使用 SDT 来实现两类重要的 SDD?
- SDT 可以在语法分析过程中实现, 但有一定限制:
 - 基本文法可以使用 LR 分析技术, 且 SDD 是 S 属性的



SDD 的实现

- 如何使用 SDT 来实现两类重要的 SDD?
- SDT 可以在语法分析过程中实现, 但有一定限制:
 - 基本文法可以使用 LR 分析技术, 且 SDD 是 S 属性的
 - 基本文法可以使用 LL 分析技术, 且 SDD 是 L 属性的



① 知识点回顾

② 语法制导翻译 (Syntax-Directed Translation, SDT)

S-SDT

L-SDT

自顶向下翻译

递归下降分析器

S-属性定义的 SDT

- **S-属性定义**：只包含综合属性。
- **综合属性**可以在分析输入符号串的同时，由**自下而上**的分析器来计算：
 - 分析栈中保存**文法符号**和有关的**综合属性**值；
 - 每当进行**归约**时，新的语法符号的属性值就由栈中正在归约的产生式右边符号的属性值来计算。

S-属性定义的 SDT

- **S-属性定义**：只包含综合属性。
- **综合属性**可以在分析输入符号串的同时，由**自下而上**的分析器来计算：
 - 分析栈中保存**文法符号**和有关的**综合属性**值；
 - 每当进行**归约**时，新的语法符号的属性值就由栈中正在归约的产生式右边符号的属性值来计算。
- 为每一个语义规则建立一个包含赋值的动作，并**把这个动作放在相应的产生式右边的末尾**。**当按照该产生式进行归约时，执行这个动作。**

S-属性定义的 SDT 示例

S-SDD 方案

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

对应的 SDT 方案

$L \rightarrow E \{ L.val = E.val \}$
 $E \rightarrow E_1 + T \{ E.val = E_1.val + T.val \}$
 $E \rightarrow T \{ E.val = T.val \}$
 $T \rightarrow T_1 * F \{ T.val = T_1.val \times F.val \}$
 $T \rightarrow F \{ T.val = F.val \}$
 $F \rightarrow (E) \{ F.val = E.val \}$
 $F \rightarrow digit \{ F.val = digit.lexval \}$

如果 S-SDD 的基本文法可以使用 LR 分析技术, 那么它的 SDT 可以在 LR 语法分析过程中实现。



如果 S-SDD 的基本文法可以使用 LR 分析技术, 那么它的 SDT 可以在 LR 语法分析过程中实现。

怎么做?



怎么做？

- 在分析栈中使用一个附加的域来存放综合属性值。
- 设语义规则 $A.a = f(X.x, Y.y, Z.z)$ 对应于产生式 $A \rightarrow XYZ$ 。

如果 S-SDD 的基本文法可以使用 LR 分析技术, 那么它的 SDT 可以在 LR 语法分析过程中实现。

怎么做?

- 在分析栈中使用一个附加的域来存放综合属性值。
- 设语义规则 $A.a = f(X.x, Y.y, Z.z)$ 对应于产生式 $A \rightarrow XYZ$ 。

归约前:

状态栈	符号	综合属性
S_0	N/A	N/A
$\vdots$	$\vdots$	$\vdots$
S_{m-2}	X	X.x
S_{m-1}	Y	Y.y
S_m	Z	Z.z

归约后:

状态栈	符号	综合属性
S_0	N/A	N/A
$\vdots$	$\vdots$	$\vdots$
S'_{m-2}	A	A.a

归约伪代码

Algorithm 1: 带语义规则 $A.a = f(X.x, Y.y, Z.z)$ 的产生式 $A \rightarrow XYZ$ 归约伪代码

- 1 $\text{stack}[\text{top}-2].\text{symb} = A;$
 - 2 $\text{stack}[\text{top}-2].\text{val} = f(\text{stack}[\text{top}-2].\text{val}, \text{stack}[\text{top}-1].\text{val}, \text{stack}[\text{top}].\text{val});$
 - 3 $\text{top} = \text{top}-2;$
-

产生式	语义动作
(1) $E' \rightarrow E$	{ print (stack[top].val); }
(2) $E \rightarrow E_1 + T$	{ stack[top-2].val = stack[top-2].val + stack[top].val; top=top-2; }
(3) $E \rightarrow T$	
(4) $T \rightarrow T_1 * F$	{ stack[top-2].val = stack[top-2].val * stack[top].val; top=top-2; }
(5) $T \rightarrow F$	
(6) $F \rightarrow (E)$	{ stack[top-2].val = stack[top-1].val; top=top-2; }
(7) $F \rightarrow digit$	

例

试分析 $3 * 5 + 4$

试分析 $3 * 5 + 4$

解

以 SLR 自动机为例

- 构造 $LR(0)$ 项集族;
- 构建 SLR 自动机;
- 算 $FOLLOW$ 集合;
- 填 $ACTION$ 和 $GOTO$ 表;
- 开始分析。

LR 分析与 S-SDT 小结

- 采用自底向上分析 (如 LR 分析), 可以首先给出 S-属性定义;



LR 分析与 S-SDT 小结

- 采用自底向上分析 (如 LR 分析), 可以首先给出 S-属性定义;
- 把 S-属性定义变成可执行的代码段, 就形成了翻译程序。

LR 分析与 S-SDT 小结

- 采用自底向上分析 (如 LR 分析), 可以首先给出 S-属性定义;
- 把 S-属性定义变成可执行的代码段, 就形成了翻译程序。
- 随着语法分析的进行, 归约前调用相应的语义子程序。

LR 分析与 S-SDT 小结

- 采用自底向上分析 (如 LR 分析), 可以首先给出 S-属性定义;
- 把 S-属性定义变成可执行的代码段, 就形成了翻译程序。
- 随着语法分析的进行, 归约前调用相应的语义子程序。
- 在这种分析模式中, 语法分析是主动的, 语义分析是从动的, 语法分析控制着语义分析。



① 知识点回顾

② 语法制导翻译 (Syntax-Directed Translation, SDT)

S-SDT

L-SDT

自顶向下翻译

递归下降分析器

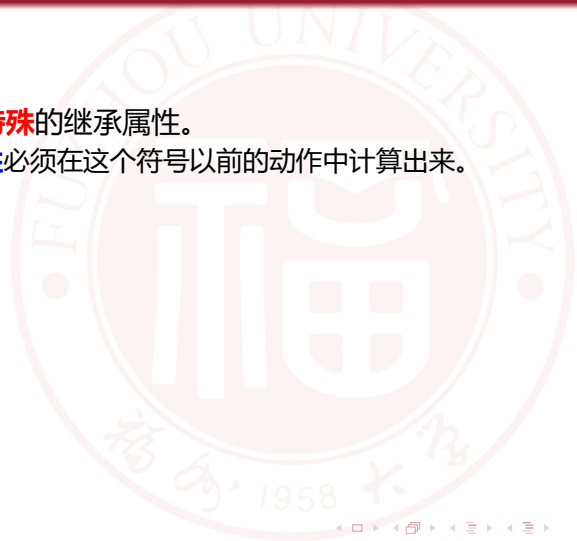
L-SDT

- **L-属性定义**：包含综合属性和**特殊**的继承属性。



L-SDT

- **L-属性定义**：包含综合属性和**特殊**的继承属性。
 - 产生式右边的符号的**继承属性**必须在这个符号以前的动作中计算出来。



L-SDT

- **L-属性定义**：包含综合属性和**特殊**的继承属性。
 - 产生式右边的符号的**继承属性**必须在这个符号以前的动作中计算出来。
 - 一个动作不能引用这个动作右边符号的**综合属性**。

L-SDT

- **L-属性定义**：包含综合属性和**特殊**的继承属性。
 - 产生式右边的符号的**继承属性**必须在这个符号以前的动作中计算出来。
 - 一个动作不能引用这个动作右边符号的**综合属性**。
 - 产生式左边非终结符号的**综合属性**只有在它所引用的所有属性都计算出来以后才能计算。计算这种属性的动作应放在产生式右端的**末尾**。

L-SDT

- **L-属性定义**：包含综合属性和**特殊**的继承属性。
 - 产生式右边的符号的**继承属性**必须在这个符号以前的动作中计算出来。
 - 一个动作不能引用这个动作右边符号的**综合属性**。
 - 产生式左边非终结符号的**综合属性**只有在它所引用的所有属性都计算出来以后才能计算。计算这种属性的动作应放在产生式右端的**末尾**。
- 转换规则：

L-SDT

- **L-属性定义**：包含综合属性和**特殊**的继承属性。
 - 产生式右边的符号的**继承属性**必须在这个符号以前的动作中计算出来。
 - 一个动作不能引用这个动作右边符号的**综合属性**。
 - 产生式左边非终结符号的**综合属性**只有在它所引用的所有属性都计算出来以后才能计算。计算这种属性的动作应放在产生式右端的**末尾**。
- 转换规则：
 - 将计算某个非终结符号 A 的**继承属性**的动作插入到产生式右部中**紧靠在 A 的本次出现之前的位置上**；

L-SDT

- **L-属性定义**：包含综合属性和**特殊**的继承属性。
 - 产生式右边的符号的**继承属性**必须在这个符号以前的动作中计算出来。
 - 一个动作不能引用这个动作右边符号的**综合属性**。
 - 产生式左边非终结符号的**综合属性**只有在它所引用的所有属性都计算出来以后才能计算。计算这种属性的动作应放在产生式右端的**末尾**。
- 转换规则：
 - 将计算某个非终结符号 A 的**继承属性**的动作插入到产生式右部中**紧靠在 A 的本次出现之前的位置上**；
 - 将计算一个产生式左部符号的**综合属性**的动作放置在这个产生式右部的**最右端**。

L-SDT

L-SDD:

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDT:

- (1) $T \rightarrow F$
- (2) $T' \rightarrow *F$
- (3) $T' \rightarrow \varepsilon$
- (4) $F \rightarrow digit$

T'
 T_1'

L-SDT

L-SDD:

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDT:

- (1) $T \rightarrow F \{ T'.inh = F.val \} T'$
- (2) $T' \rightarrow *F \quad T'_1$
- (3) $T' \rightarrow \varepsilon$
- (4) $F \rightarrow digit$

L-SDT

L-SDD:

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDT:

- (1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
- (2) $T' \rightarrow *F \quad T'_1$
- (3) $T' \rightarrow \varepsilon$
- (4) $F \rightarrow digit$

L-SDT

L-SDD:

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDT:

- (1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
- (2) $T' \rightarrow *F \{ T'_1.inh = T'.inh * F.val \} T'_1$
- (3) $T' \rightarrow \varepsilon$
- (4) $F \rightarrow digit$

L-SDT

L-SDD:

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDT:

- (1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
- (2) $T' \rightarrow *F \{ T'_1.inh = T'.inh * F.val \} T'_1 \{ T'.syn = T'_1.syn \}$
- (3) $T' \rightarrow \varepsilon$
- (4) $F \rightarrow digit$

L-SDT

L-SDD:

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDT:

- (1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
- (2) $T' \rightarrow *F \{ T'_1.inh = T'.inh * F.val \} T'_1 \{ T'.syn = T'_1.syn \}$
- (3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- (4) $F \rightarrow digit$

L-SDT

L-SDD:

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDT:

- (1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
- (2) $T' \rightarrow *F \{ T'_1.inh = T'.inh * F.val \} T'_1 \{ T'.syn = T'_1.syn \}$
- (3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- (4) $F \rightarrow digit \{ F.val = digit.lexval \}$

例

请指出下列语义动作中哪个是有问题的：

- $S \rightarrow A1 A2 \{A1.in = 1; A2.in = 2\}$
- $S \rightarrow \{A1.in = 1; A2.in = 2\} A1 A2$
- $S \rightarrow \{A1.in = 1\} A1 \{A2.in = 2\} A2$

例

请指出下列语义动作中哪个是有问题的：

- $S \rightarrow A1\ A2\ \{A1.in = 1; A2.in = 2\}$
- $S \rightarrow \{A1.in = 1; A2.in = 2\}\ A1\ A2$
- $S \rightarrow \{A1.in = 1\}\ A1\ \{A2.in = 2\}\ A2$

例

请指出下列语义动作中哪个是有问题的：

- $S \rightarrow A1 A2 \{A1.in = 1; A2.in = 2\}$
- $S \rightarrow \{A1.in = 1; A2.in = 2\} A1 A2$
- $S \rightarrow \{A1.in = 1\} A1 \{A2.in = 2\} A2$

随堂练习

例 (写出如下 SDD 的 SDT 语义动作)

产生式	语义规则
$S \rightarrow B$	$B.ps = 10$ $S.ht = B.ht$
$B \rightarrow B_1 B_2$	$B_1.ps = B.ps$ $B_2.ps = B.ps$ $B.ht = \max(B_1.ht, B_2.ht)$
$B \rightarrow B_1 \text{sub} B_2$	$B_1.ps = B.ps$ $B_2.ps = \text{shrink}(B.ps)$ $B.ht = \text{disp}(B_1.ht, B_2.ht)$
$B \rightarrow \text{text}$	$B.ht = \text{text.h} \times B.ps$

解

$$\begin{aligned}
 S &\rightarrow \{ B.ps = 10 \} B \{ S.ht = B.ht \} \\
 B &\rightarrow \{ B1.ps = B.ps \} B1 \{ B2.ps = B.ps \} B2 \{ B.ht = \max(B1.ht, B2.ht) \} \\
 B &\rightarrow \{ B1.ps = B.ps \} B1 \text{ sub } \{ B2.ps = \text{shrink}(B.ps) \} B2 \\
 &\quad \{ B.ht = \text{disp}(B1.ht, B2.ht) \} \\
 B &\rightarrow \text{text} \{ B.ht = \text{text.h} \times B.ps \}
 \end{aligned}$$

如果一个 L-SDD 的基本文法可以使用 LL 分析技术, 那么它的 SDT 可以在 **LL 或 LR** 语法分析过程中实现。



① 知识点回顾

② 语法制导翻译 (Syntax-Directed Translation, SDT)

S-SDT

L-SDT

自顶向下翻译

递归下降分析器

自顶向下的翻译

- 自顶向下的翻译过程中，语义动作是在处于相同位置的符号**被展开（或匹配成功）**时执行的。
- 为了构造不带回溯的自顶向下语法分析，必须消除文法中的左递归，提取左公因子。

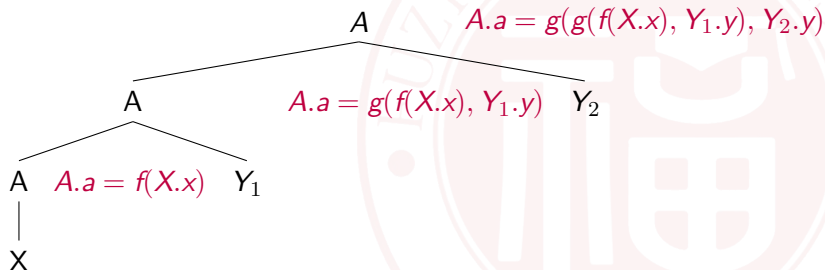
消除左递归的一般方法

- 左递归文法不能按照自顶向下方式确定地进行语法分析，当文法是 SDT 的一部分的时候，在**消除文法左递归的同时还需要考虑如何处理其中的语义动作。**

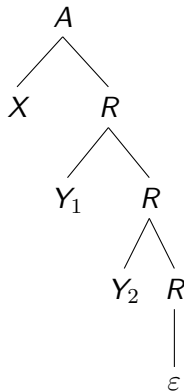
例 (带有文法左递归的 SDT)

- $A \rightarrow A1 \ Y \ \{ \ A.a = g(A1.a, Y.y) \}$
- $A \rightarrow X \ \{ \ A.a = f(X.x) \}$ 其中， g 和 f 是任意函数

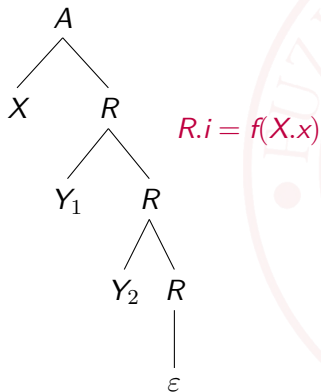
例 (带文法左递归的注释分析树)



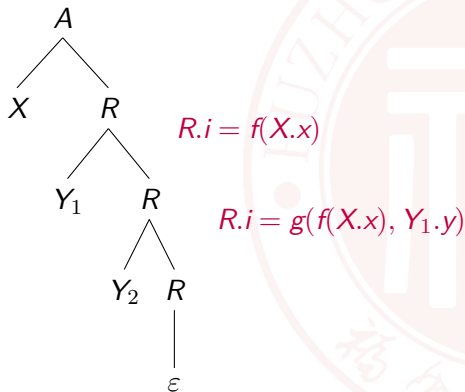
例 (消除文法左递归的注释分析树 (XYY 为例))



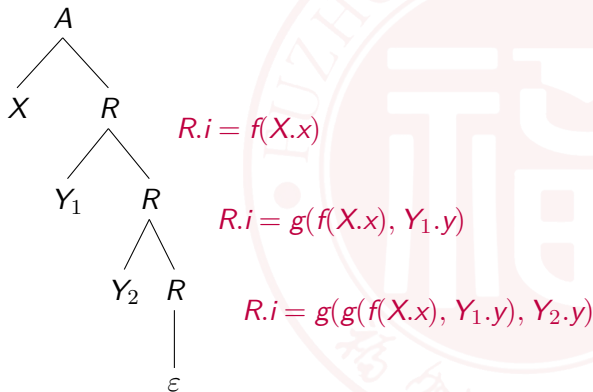
例 (消除文法左递归的注释分析树 (XYY 为例))



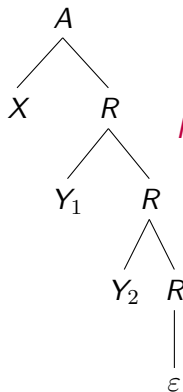
例 (消除文法左递归的注释分析树 (XYY 为例))



例 (消除文法左递归的注释分析树 (XYY 为例))



例 (消除文法左递归的注释分析树 (XYY 为例))



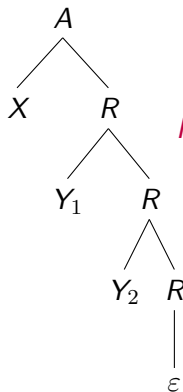
$$R.i = f(X.x)$$

$$R.i = g(f(X.x), Y_1.y)$$

$$R.s = g(g(f(X.x), Y_1.y), Y_2.y)$$

$$R.i = g(g(f(X.x), Y_1.y), Y_2.y)$$

例 (消除文法左递归的注释分析树 (XYY 为例))



$$R.i = f(X.x)$$

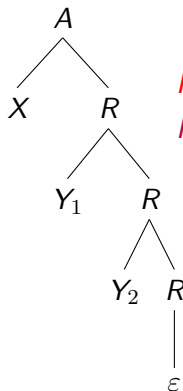
$$R.s = g(g(f(X.x), Y_1.y), Y_2.y)$$

$$R.i = g(f(X.x), Y_1.y)$$

$$R.s = g(g(f(X.x), Y_1.y), Y_2.y)$$

$$R.i = g(g(f(X.x), Y_1.y), Y_2.y)$$

例 (消除文法左递归的注释分析树 (XYY 为例))



$$R.s = g(g(f(X.x), Y_1.y), Y_2.y)$$

$$R.i = f(X.x)$$

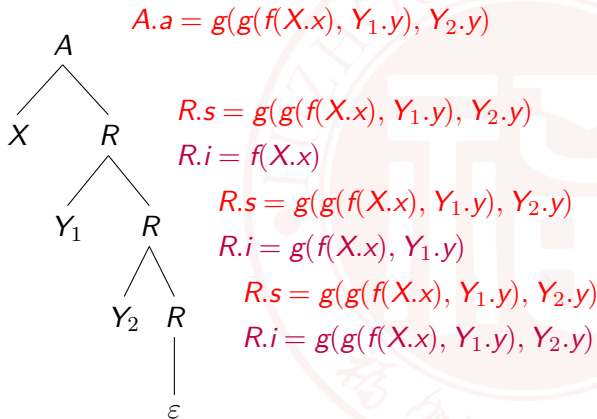
$$R.s = g(g(f(X.x), Y_1.y), Y_2.y)$$

$$R.i = g(f(X.x), Y_1.y)$$

$$R.s = g(g(f(X.x), Y_1.y), Y_2.y)$$

$$R.i = g(g(f(X.x), Y_1.y), Y_2.y)$$

例 (消除文法左递归的注释分析树 (XYY 为例))



消除左递归的一般方法

- 左递归文法不能按照自顶向下方式确定地进行语法分析，当文法是 SDT 的一部分的时候，在**消除文法左递归的同时还需要考虑如何处理其中的语义动作**。

例 (带有文法左递归的 SDT)

- $A \rightarrow A1 \ Y \ \{ \ A.a = g(A1.a, Y.y) \}$
- $A \rightarrow X \ A.a = f(X.x)$ 其中， g 和 f 是任意函数

基础文法消除左递归：

- $A \rightarrow XR$
- $R \rightarrow YR \mid \varepsilon$

翻译模式消除左递归：

- $A \rightarrow X \ \{ R.i = f(X.x) \} \ R \ \{ A.a = R.s \}$
- $R \rightarrow Y \ \{ R1.i = g(R.i, Y.y) \} \ R1 \ \{ R.s = R1.s \}$
- $R \rightarrow \varepsilon \ \{ R.s = R.i \}$

综合示例

例

以下是关于算术表达式的左递归文法和相应的翻译模式，请构建非左递归文法及其翻译模式并计算 $9 - 5 + 2$ 。

$$\begin{aligned} E &\rightarrow E1 + T \{E.val = E1.val + T.val\} \\ E &\rightarrow E1 - T \{E.val = E1.val - T.val\} \\ E &\rightarrow T \{E.val = T.val\} \\ T &\rightarrow (E) \{T.val = E.val\} \\ T &\rightarrow num \{T.val = num.val\} \end{aligned}$$



① 知识点回顾

② 语法制导翻译 (Syntax-Directed Translation, SDT)

S-SDT

L-SDT

自顶向下翻译

递归下降分析器

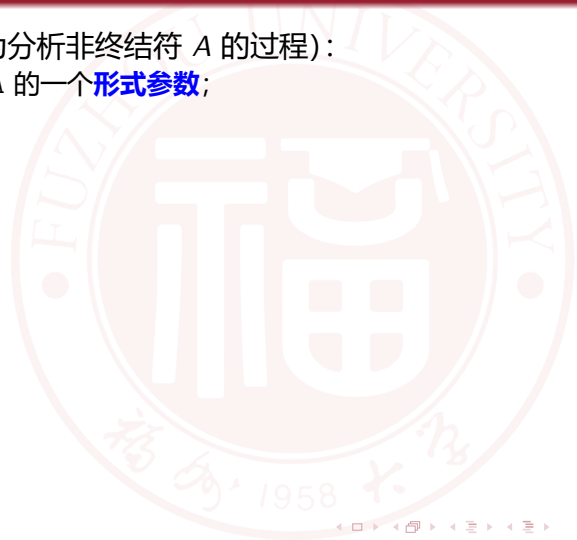
递归下降分析器的设计

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):



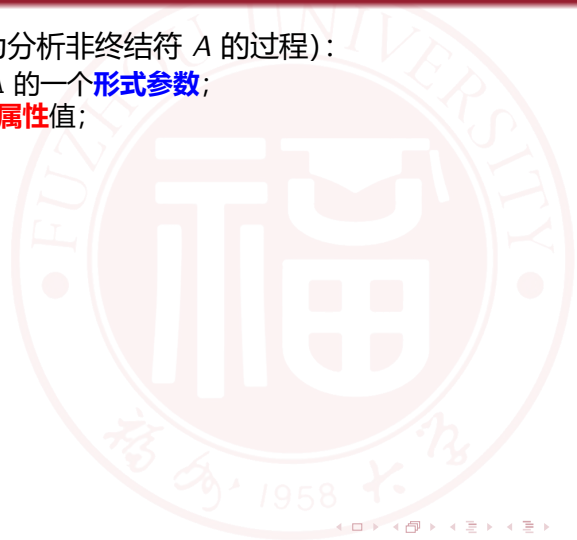
递归下降分析器的设计

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):
 - A 的每个**继承属性**对应函数 A 的一个**形式参数**;



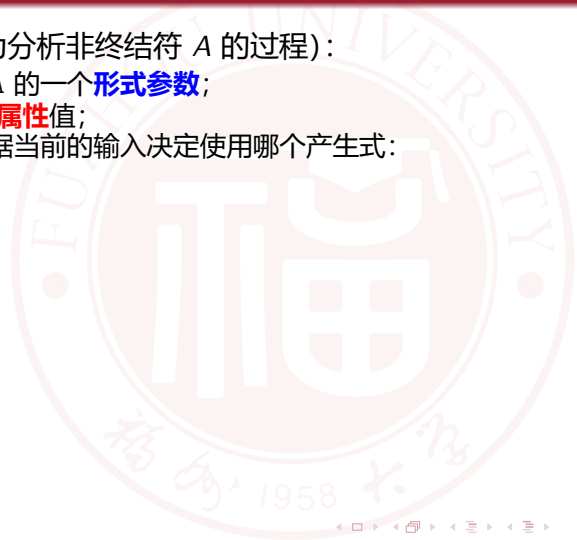
递归下降分析器的设计

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):
 - A 的每个**继承属性**对应函数 A 的一个**形式参数**;
 - 函数 A 的**返回值**为 A 的**综合属性**值;



递归下降分析器的设计

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):
 - A 的每个**继承属性**对应函数 A 的一个**形式参数**;
 - 函数 A 的**返回值**为 A 的**综合属性**值;
 - 函数 A 的函数体的作用是根据当前的输入决定使用哪个产生式:



递归下降分析器的设计

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):
 - A 的每个**继承属性**对应函数 A 的一个**形式参数**;
 - 函数 A 的**返回值**为 A 的**综合属性**值;
 - 函数 A 的函数体的作用是根据当前的输入决定使用哪个产生式:
 - 对出现在 A 产生式中的**每个**文法符号的**每个****属性**都设置一个**局部变量**;

分析非终结符 A 的过程):

- 的一个**形式参数**;
- 属性**值;
- 据当前的输入决定使用哪个产生式:
- 个文法符号的**每个属性**都设置一个**局部变量**;
- 其它变量**。

- 分析非终结符 A 的过程):
- 的一个**形式参数**;
 - 属性**值;
 - 据当前的输入决定使用哪个产生式:
 - 个文法符号的**每个属性**都设置一个**局部变量**;
 - 其它变量**。

递归下降分析器的设计

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):
 - A 的每个**继承属性**对应函数 A 的一个**形式参数**;
 - 函数 A 的**返回值**为 A 的**综合属性**值;
 - 函数 A 的函数体的作用是根据当前的输入决定使用哪个产生式:
 - 对出现在 A 产生式中的**每个**文法符号的**每个****属性**都设置一个**局部变量**;
 - 为了计算这些属性值所需的**其它变量**。
- 按照产生式右部从左到右的, 对于**单词符号 (终结符)**、**非终结符**和**语义动作**, 分别实现

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):
 - A 的每个**继承属性**对应函数 A 的一个**形式参数**;
 - 函数 A 的**返回值**为 A 的**综合属性值**;
 - 函数 A 的函数体的作用是根据当前的输入决定使用哪个产生式:
 - 对出现在 A 产生式中的**每个**文法符号的**每个属性**都设置一个**局部变量**;
 - 为了计算这些属性值所需的**其它变量**。
- 按照产生式右部从左到右的, 对于**单词符号 (终结符)**、**非终结符**和**语义动作**, 分别实现
 - 对于带有综合属性 x 的**终结符** X , 把 x 的值保存在**局部变量** $X.x$ 中; 然后产生一个匹配 X 的调用, 并继续输入;

递归下降分析器的设计

- 扩展分析器 (设函数 $A(\dots)$ 为分析非终结符 A 的过程):
 - A 的每个**继承属性**对应函数 A 的一个**形式参数**;
 - 函数 A 的**返回值**为 A 的**综合属性**值;
 - 函数 A 的函数体的作用是根据当前的输入决定使用哪个产生式:
 - 对出现在 A 产生式中的**每个**文法符号的**每个属性**都设置一个**局部变量**;
 - 为了计算这些属性值所需的**其它变量**。
- 按照产生式右部从左到右的, 对于**单词符号 (终结符)**、**非终结符**和**语义动作**, 分别实现
 - 对于带有综合属性 x 的**终结符** X , 把 x 的值保存在**局部变量** $X.x$ 中; 然后产生一个匹配 X 的调用, 并继续输入;
 - 对于**非终结符** B , 产生一个右部带有函数调用的赋值语句 $c = B(b_1, b_2, \dots, b_k)$, 其中, $b_1, b_2, \dots, b_k$ 是代表 B 的**继承属性**的变量, c 是代表 B 的**综合属性**的变量;

- 

while 循环的 SDT 翻译方案

例 (将 while 语句转换成中间代码 (字符串表示) 的递归下降分析)

假设程序文法部分如下:

$$\begin{aligned} P &\rightarrow SP' \\ S &\rightarrow \text{while } (C) S_1 \end{aligned}$$

- 设置标号: 控制程序的走向
- 指令 Label L: 标明后一条指令的标号为 L。

while 循环的 SDT 翻译方案

例 (将 while 语句转换成中间代码 (字符串表示) 的递归下降分析)

- 设置属性：
 - 继承属性 S.next: S 之后执行的代码的开始处的标号;
 - 综合属性 S.code: 实现语句 S 的中间代码序列, 最后有一条跳转到 S.next 的指令;
 - 继承属性 C.true: C 为真时执行的代码的开始处的标号;
 - 继承属性 C.false: C 为假时执行的代码的开始处的标号;
 - 综合属性 C.code: 实现条件表达式 C 的计算的中间代码序列, 并根据 C 的值跳转到 C.true 或 C.false;

写出 SDD

写出 SDT

